Relazione tecnica per il corso di

Tecnologie del linguaggio naturale

Convertitore Lessicale Italiano → Yodish

Parsing con CKY e trasformazione dell'ordine delle parole XSV -> SVX

Olivero Alessandro
Franco Andrea
Iudice Samuele

Università degli Studi di Torino

Corso di Laurea Magistrale in Informatica

Anno Accademico 2025/26

Sommario

Il presente elaborato descrive la progettazione e l'implementazione di un sistema per la conversione automatica di frasi dalla lingua italiana in lingua Yodish, variante della lingua italiana che riproduce il caratteristico ordine sintattico utilizzato da Yoda nella saga di Guerre Stellari. Il sistema si basa su un parser CKY implementato in Python e su un modulo di trasformazione ricorsiva dell'albero di derivazione. Vengono analizzati l'intero workflow progettuale, dalla modellazione della grammatica alla manipolazione sintattica, discutendo i risultati ottenuti e i limiti intrinseci dell'approccio basato su regole.

Indice

1 Introduzione **3**

2 Teoria applicata **3**
2.1 Grammatiche Context-Free (CFG) 3
2.2 Forma Normale di Chomsky (CNF) 4
2.3 L'Algoritmo CKY 4

3 Struttura del Progetto **4**
3.1 Il Lessico (`Lexicon.json`) 5
3.2 Le Regole Grammaticali (`regole_binarie.json`) 5

4 Implementazione **6**
4.1 La Classe `Nodo` 6
4.2 L'Algoritmo CKY: Implementazione 7
4.3 La Trasformazione in Ordine Yoda 7
4.4 Linearizzazione e Post-processing 8

5 Interfaccia da Linea di Comando (CLI) **8**

6 Esempio di Esecuzione **9**
6.1 Albero di Derivazione Originale 9
6.2 Albero Trasformato in Ordine Yoda 10

7 Limiti e Possibili Estensioni **10**
7.1 Limiti del Sistema Attuale 10
7.2 Possibili Estensioni 11

8 Confronto con Approcci Alternativi **11**
8.1 Parser Rule-Based vs. Neurali 11
8.2 Trasformazione via Regole vs. Modelli Seq2Seq 12

9 Conclusioni **12**

1 Introduzione

L'obiettivo è creare un sistema di conversione sintattica dall'italiano standard **SVX** all'ordine **XSV** tipico del personaggio Yoda. Sviluppato in Python senza dipendenze esterne, il software modella il linguaggio tramite grammatiche context-free (CFG) in Forma Normale di Chomsky (CNF). Il processo computazionale si articola in tre fasi principali:

- **Parsing CKY**: analisi bottom-up della frase tramite un lessico e regole grammaticali definiti da file in formato JSON.
- **Trasformazione**: manipolazione ricorsiva dell'albero di derivazione per il riposizionamento dei costituenti sintattici.
- **Linearizzazione**: generazione della stringa finale a partire dalle foglie dell'albero trasformato.

La scelta di separare algoritmi e conoscenze linguistiche dona al sistema maggiore modularità e portabilità.

2 Teoria applicata

2.1 Grammatiche Context-Free (CFG)

Una **grammatica context-free** è una quadrupla $G = (V, \Sigma, R, S)$ dove:

- V è un insieme finito di simboli non-terminali, tipicamente rappresentanti categorie sintattiche come NP, VP, S;
- Σ è un insieme finito di terminali, con $V \cap \Sigma = \emptyset$;
- R è un insieme finito di regole di produzione della forma $A \to \alpha$, con $A \in V$ e $\alpha \in (V \cup \Sigma)^*$;
- $S \in V$ è il simbolo iniziale (assioma).

Una stringa $w \in \Sigma^*$ è generata da G se esiste una derivazione $S \Rightarrow^* w$. Il linguaggio $L(G)$ generato dalla grammatica è l'insieme di tutte le stringhe derivabili dall'assioma.

Le CFG sono ampiamente usate in linguistica computazionale per descrivere la struttura sintattica delle frasi.

2.2 Forma Normale di Chomsky (CNF)

Un algoritmo di parsing efficiente come CKY richiede che la grammatica sia in **Forma Normale di Chomsky** (CNF). Una CFG è in CNF se tutte le regole di produzione sono della forma:

$$A \to BC \quad \text{oppure} \quad A \to a$$

dove $A, B, C \in V$ e $a \in \Sigma$. In altre parole, ogni regola produce esattamente due non-terminali oppure un singolo terminale.

Qualsiasi CFG senza ε-produzioni può essere convertita in CNF attraverso i passi standard di *binarizzazione* delle regole con più di due simboli nella parte destra e *eliminazione delle regole unitarie*.

Nel progetto, la grammatica è stata progettata direttamente in CNF; le regole lessicali (terminali) sono memorizzate nel lessico, mentre le regole binarie $A \to BC$ sono nel file `regole_binarie.json`.

2.3 L'Algoritmo CKY

L'algoritmo **CKY** (da Cocke, Kasami e Younger, 1965–1969) è un algoritmo di parsing bottom-up per grammatiche in CNF, basato su programmazione dinamica.

Dato l'input $w = w_1 w_2 \cdots w_n$, l'algoritmo costruisce una *tabella triangolare* T di dimensione $n \times n$, dove la cella $T[i][j]$ contiene l'insieme dei non-terminali che possono derivare la sottostringa $w_i w_{i+1} \cdots w_j$.

Algoritmo:

1. **Inizializzazione**: per ogni i da 1 a n, inserire in $T[i][i]$ tutti i non-terminali A tali che $A \to w_i$ è una regola lessicale.

2. **Riempimento**: per ogni lunghezza di span ℓ da 2 a n, per ogni inizio i e fine $j = i + \ell - 1$, per ogni punto di taglio k da i a $j - 1$: se $B \in T[i][k]$ e $C \in T[k+1][j]$ e $A \to BC \in R$, allora inserire A in $T[i][j]$.

3. **Accettazione**: la frase è riconosciuta se e solo se $S \in T[1][n]$.

La complessità temporale è $O(n^3 \cdot |G|)$ dove $|G|$ è la dimensione della grammatica. La variante con ricostruzione dell'albero (usata in questo progetto) memorizza, per ogni cella, anche i puntatori ai sottoalberi che hanno giustificato l'inserimento di un non-terminale.

3 Struttura del Progetto

Il progetto è organizzato in tre componenti principali:

Tabella 1: Componenti del sistema IT-YO

File	Tipo	Descrizione
`yoda.py`	Codice Python	Modulo principale: parsing, trasformazione, CLI
`Lexicon.json`	Dati JSON	Lessico italiano con etichette POS
`regole_binarie.json`	Dati JSON	Regole binarie della grammatica in CNF

3.1 Il Lessico (`Lexicon.json`)

Il lessico è un dizionario JSON che mappa ogni parola italiana a una lista di categorie grammaticali (POS tag). Ad esempio:

```
{
  "Mario":    ["N", "NP"],
  "gatto":    ["N"],
  "mangia":   ["V"],
  "il":       ["Det"],
  "lentamente": ["Adv"],
  "novecento": ["Num"]
}
```

Listing 1: Esempio di voci nel lessico

Le categorie utilizzate nella grammatica sono: `Det` (determinante), `N` (nome), `NP` (nome proprio / sintagma nominale pre-formato), `V` (verbo), `Adj` (aggettivo), `Adv` (avverbio), `Prep` (preposizione), `Num` (numerale).

3.2 Le Regole Grammaticali (`regole_binarie.json`)

Il file contiene le regole binarie della grammatica in CNF, dove la chiave è la coppia di figli e il valore è la lista dei possibili genitori. La scelta di *invertire* la direzione rispetto alla notazione tradizionale (dai figli al genitore) è una ottimizzazione per l'algoritmo CKY: durante il parsing si cerca quali genitori sono compatibili con una coppia di figli già riconosciuti, non il contrario.

```
{
  "Det,N":    ["NP"],
  "NP,VP":    ["S"],
  "V,NP":     ["VP", "VP1"],
  "VP1,Adv":  ["VP"],
  "Prep,NP":  ["PP"],
```

```
  "V,PP":     ["VP"],
  "V,Adj":    ["VP"],
  "Num,N":    ["NP"]
}
```

Listing 2: Esempio di regole binarie (formato chiave-valore invertito)

La funzione `carica_dizionari()` in fase di inizializzazione converte le chiavi stringa (es. `"Det,N"`) nelle tuple Python richieste dall'algoritmo (es. `('Det', 'N')`).

4 Implementazione

4.1 La Classe `Nodo`

L'albero di derivazione è rappresentato tramite la classe `Nodo`, che implementa un nodo di albero n-ario con tre attributi:

- `label`: il simbolo grammaticale del nodo (es. `'NP'`, `'VP'`) oppure la parola stessa nelle foglie.
- `children`: lista di nodi figli.
- `word`: la parola (solo per le foglie lessicali).

I nodi foglia (terminali) sono distinti da quelli interni tramite il metodo `is_leaf()`, che verifica se la lista `children` è vuota.

La struttura è volutamente semplice per facilitare la visita ricorsiva sia nella stampa (`pretty_print`) che nella trasformazione Yoda.

```python
class Nodo:
    def __init__(self, label, children=None, word=None):
        self.label    = label
        self.children = children or []
        self.word     = word

    def is_leaf(self):
        return len(self.children) == 0
```

Listing 3: Classe Nodo – costruttore e metodi principali

Il metodo `pretty_print()` produce una rappresentazione visuale ad albero nel terminale, con connettori grafici ASCII (├──, └──, │) analoga a quella del comando Unix `tree`.

4.2 L'Algoritmo CKY: Implementazione

La funzione `cky_parse(sentence)` riceve una lista di parole tokenizzate e restituisce il nodo radice dell'albero di derivazione (etichettato `'S'`) oppure `None` se la frase non è riconosciuta dalla grammatica.

L'implementazione usa una tabella tridimensionale `table[i][j]` dove ogni cella è un dizionario `{simbolo: Nodo}`. Questa scelta — memorizzare direttamente il nodo invece del semplice booleano — consente di ricostruire l'albero durante il parsing senza una fase separata di *backtracking*.

```
for span in range(2, n + 1):
    for i in range(n - span + 1):
        j = i + span - 1
        for k in range(i, j):
            for (B, C), parents in binary_rules.items():
                if B in table[i][k] and C in table[k+1][j]:
                    for A in parents:
                        if A not in table[i][j]:
                            node_B = table[i][k][B]
                            node_C = table[k+1][j][C]
                            table[i][j][A] = Nodo(
                                label=A,
                                children=[node_B, node_C]
                            )
```

Listing 4: Nucleo dell'algoritmo CKY con ricostruzione dell'albero

Si noti che, in caso di ambiguità grammaticale, l'algoritmo mantiene solo il *primo* albero trovato per ogni non-terminale in ogni cella, corrispondente alla prima regola applicabile nell'ordine di iterazione del dizionario. Una grammatica ambigua produce risultati deterministici ma dipendenti dall'ordinamento delle regole.

4.3 La Trasformazione in Ordine Yoda

Il modulo più originale del progetto è la funzione `trasforma_in_yoda()`, che visita l'albero in post-ordine (prima i figli, poi il nodo corrente) e riorganizza i sottoalberi laddove trova la struttura sintattica canonica italiana.

La regola principale è: ogni nodo `S` con struttura $S \rightarrow NP\ VP$ viene trasformato in $S_yoda \rightarrow Comp\ NP\ V$, dove `Comp` è il complemento contenuto nel VP. In altre parole, il complemento viene spostato in posizione iniziale di frase.

Sono gestiti tre casi distinti:

Caso 1: Frase transitiva semplice $VP \to V\ X$ con $X \in \{NP, PP, Adj\}$

$$S \to NP\ (VP \to V\ Comp) \quad \Longrightarrow \quad S_yoda \to Comp\ NP\ V$$

Caso 2: Frase con avverbio $VP \to VP1\ Adv$ con $VP1 \to V\ NP$

$$S \to NP\ (VP \to (VP1 \to V\ NP)\ Adv) \quad \Longrightarrow \quad S_yoda \to NP\ Sogg\ V\ Adv$$

Caso 3: Frase intransitiva $VP \to V$ (solo verbo)

$$S \to NP\ (VP \to V) \quad \Longrightarrow \quad S_yoda \to NP\ V$$

(nessuna trasposizione, l'ordine resta invariato poiché non c'è complemento da anteporre)

```
if child0.label == 'V' and child1.label in ('NP', 'PP', 'Adj'):
    verbo = child0
    compl = child1
    nodo.label = 'S_yoda'
    nodo.children = [compl, sogg, verbo]
```

Listing 5: Trasformazione Caso 1 – VP con complemento diretto

4.4 Linearizzazione e Post-processing

La funzione `raccogli_foglie()` effettua una visita DFS (depth-first, in-order) sull'albero trasformato, raccogliendo le parole nelle foglie da sinistra a destra. La funzione `converti_in_yoda()` applica infine un post-processing minimo:

- Le parole vengono rese minuscole, salvo i *nomi propri*, che vengono identificati come parole che appaiono nel lessico con entrambi i tag `N` e `NP` e iniziano con la maiuscola.
- La prima parola della frase Yoda viene capitalizzata.

5 Interfaccia da Linea di Comando (CLI)

Il modulo è utilizzabile direttamente da terminale grazie al modulo `argparse` della libreria standard Python. Sono supportate due modalità d'uso:

Frase singola:

```
python yoda.py "Mario mangia il gatto"
```

Modalità test:

```
python yoda.py --test
```

La modalità `-test` esegue una suite di frasi predefinite confrontando l'output ottenuto con quello atteso. Ogni test riporta `CORRETTO` o `ERRORE` con il dettaglio dell'output ottenuto vs. atteso.

Le tre frasi di test incluse nel codice sono:

Tabella 2: Suite di test del modulo `-test`

Input italiano	**Output Yoda atteso**
Tu hai amici lì	Amici tu hai lì
Tu avrai novecento anni di età	Novecento anni di età tu avrai
Noi siamo illuminati	Illuminati noi siamo

L'output a schermo include, per ogni frase: la frase originale, l'albero di derivazione iniziale, l'albero dopo la trasformazione Yoda, e la frase finale. Questo livello di verbosità è intenzionale a scopo didattico.

6 Esempio di Esecuzione

Per illustrare concretamente il funzionamento del sistema, si considera la frase di input *"Tu hai amici lì"*.

6.1 Albero di Derivazione Originale

La frase viene tokenizzata in `[Tu, hai, amici, lì]`. Il parser CKY riconosce la struttura seguente:

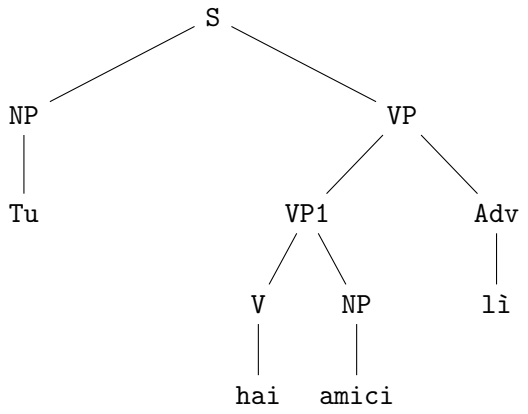

Figura 1: Albero di derivazione della frase "Tu hai amici lì"

6.2 Albero Trasformato in Ordine Yoda

La funzione `trasforma_in_yoda()` identifica la struttura $S \rightarrow NP\ (VP \rightarrow (VP1 \rightarrow V\ NP)\ Adv)$ (Caso 2) e riorganizza le costituenti nell'ordine *Comp Sogg V Adv*:

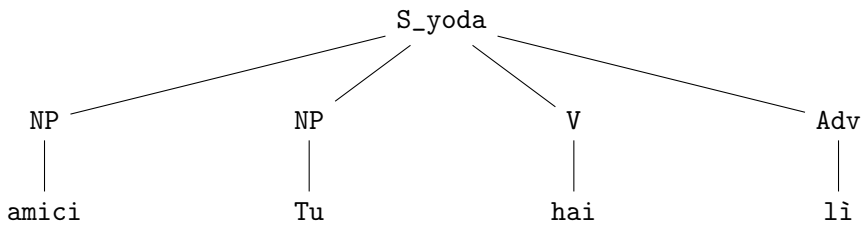

Figura 2: Albero trasformato in ordine Yoda: "Amici tu hai lì"

La linearizzazione delle foglie produce la frase finale:

Input: *Tu hai amici lì*

Output IT-YO: *Amici tu hai lì*

7 Limiti e Possibili Estensioni

7.1 Limiti del Sistema Attuale

Copertura lessicale limitata. Il lessico comprende un numero ridotto di voci, pensato per le frasi di test. L'aggiunta di nuove frasi richiede l'estensione manuale del file `Lexicon.json`, il che non è scalabile per frasi arbitrarie.

Grammatica ristretta. La grammatica in CNF copre costruzioni sintattiche semplici (frasi semplici SVO con al più un avverbio o un sintagma preposizionale). Non sono gestite: subordinate, relative, coordinate, frasi passive, costruzioni con clitici (es. *lo vedo*), né strutture con ausiliari complessi.

Ambiguità non gestita esplicitamente. In caso di ambiguità strutturale (più alberi possibili), viene restituito solo il primo albero trovato nell'ordine di iterazione del dizionario delle regole. Il sistema non avvisa l'utente della presenza di ambiguità, né consente di esplorare parsing alternativi.

Assenza di analisi morfologica. Il lessico è basato sulla forma di superficie delle parole: forme flesse diverse dello stesso lemma (es. *mangia*, *mangiano*, *mangiava*) devono essere inserite separatamente. Non è presente alcun modulo di lemmatizzazione o analisi morfologica.

Trasformazione solo al livello radice. La funzione `trasforma_in_yoda()` agisce ricorsivamente, ma la trasformazione XSV è applicata solo ai nodi `S` di primo livello. Le frasi subordinate annidate non vengono trasformate.

7.2 Possibili Estensioni

- **Estensione del lessico tramite risorse esterne**: integrazione con WordNet italiano o PoS tagger preaddestrati (es. spaCy, Stanza) per ottenere la copertura su frasi arbitrarie.
- **Grammatica più ricca**: aggiunta di regole per subordinate relative, completive, coordinate, costruzioni passive e perifrasi verbali.
- **Parser probabilistico (PCFG + CKY)**: associare probabilità alle regole per gestire l'ambiguità in modo principiato, selezionando l'albero più probabile (algoritmo CKY probabilistico con massimizzazione del prodotto delle probabilità).
- **Interfaccia grafica o web**: creazione di una semplice interfaccia web (es. Flask + HTML) o di una GUI per rendere il sistema accessibile a utenti non tecnici.
- **Supporto a più trasformazioni**: oltre all'ordine Yoda, si potrebbero implementare altre trasformazioni sintattiche (topicalizzazione, dislocazione a sinistra, passivizzazione) come varianti configurabili.

8 Confronto con Approcci Alternativi

8.1 Parser Rule-Based vs. Neurali

Il parser implementato in questo progetto è di tipo **rule-based simbolico**: le regole grammaticali sono scritte esplicitamente dall'autore, il lessico è compilato a mano, e l'algoritmo CKY produce alberi deterministici in base alla grammatica fornita. Questo approccio ha il vantaggio della piena interpretabilità (ogni decisione è tracciabile nelle regole) e del controllo totale sulla struttura degli alberi prodotti.

In contrasto, i moderni parser neurali (es. quelli basati su BERT, XLM-R o architetture Transformer encoder-decoder) apprendono le strutture sintattiche da grandi corpora annotati (es. Universal Dependencies) e raggiungono prestazioni molto superiori su frasi naturali non vincolate. Tuttavia, questi modelli:

- Non sono interpretabili in modo diretto;
- Richiedono risorse computazionali e dati di addestramento significativi;
- Non garantiscono alberi che rispettino una grammatica formale definita dall'utente.

Per gli obiettivi didattici di questo progetto — comprendere la struttura degli algoritmi di parsing e manipolare direttamente gli alberi sintattici — l'approccio simbolico è quello più adeguato.

8.2 Trasformazione via Regole vs. Modelli Seq2Seq

La trasformazione da ordine SVO a XSV è qui implementata come una semplice visita e riorganizzazione dell'albero. Un approccio alternativo è l'uso di modelli *sequence-to-sequence* addestrati su coppie di frasi (italiano → italiano-Yoda), che apprendono implicitamente la trasformazione senza una rappresentazione intermedia esplicita. Ancora una volta, il vantaggio dell'approccio strutturale adottato è la trasparenza e la garanzia formale che l'output rispetti le trasformazioni progettate.

9 Conclusioni

Il progetto IT-YO ha consentito di mettere in pratica i concetti fondamentali di analisi sintattica formale: progettazione di una grammatica context-free in Forma Normale di Chomsky, implementazione dell'algoritmo CKY con ricostruzione dell'albero di derivazione, e manipolazione strutturale dell'albero per ottenere una trasformazione dell'ordine delle parole.

Il sistema è funzionante per un sottoinsieme controllato di frasi italiane e produce output corretti sui casi di test previsti. La separazione tra grammatica (dati JSON) e logica di parsing (codice Python) rende il sistema facilmente estendibile: per coprire nuove costruzioni sintattiche è sufficiente aggiungere voci al lessico e regole al file JSON, senza modificare l'algoritmo.

Dal punto di vista didattico, il progetto illustra concretamente come la teoria delle grammatiche formali si traduca in un sistema computazionale funzionante, e come la manipolazione degli alberi di derivazione sia uno strumento potente per operare trasformazioni linguistiche strutturate.

Tra i possibili sviluppi futuri, l'integrazione con un PoS tagger automatico e l'estensione della grammatica a costruzioni più complesse rappresentano le direzioni più promettenti per avvicinare il sistema a un uso su frasi naturali non vincolate.

Riferimenti bibliografici

[1] Chomsky, N. (1965). *Aspects of the Theory of Syntax*. MIT Press, Cambridge, MA.

[2] Cocke, J. (1969). Programming languages and their compilers: Preliminary notes. *Courant Institute of Mathematical Sciences*, New York University.

[3] Kasami, T. (1966). An efficient recognition and syntax-analysis algorithm for context-free languages. *Technical Report AFCRL-65-758*, Air Force Cambridge Research Laboratory.

[4] Younger, D. H. (1967). Recognition and parsing of context-free languages in time n^3. *Information and Control*, 10(2), 189–208.

[5] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft). Disponibile online: `https://web.stanford.edu/~jurafsky/slp3/`

[6] Manning, C. D., & Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press.

[7] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Addison-Wesley.